

Messages, roles, and multi-turn context

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN

Session 7 of 12 · English notes

A single `messages.create` call is one HTTP round trip. Claude does not remember the previous turn unless **you send that history again**. Vijay's notebooks built a list of `{role, content}` objects, appended the assistant's last text, then asked a follow-up. That list is the conversational **context** your application owns.

Roles: user, assistant, and system

Class language mapped **user** to the human message and **assistant** to model output you replay. Those two live in the `messages` array. **System** is not a third role inside that array in the pattern Vijay coded: it is a **separate parameter** on `create` (the `system` field). Exam wording he flagged: they will ask how messages and the system prompt are configured. System is instructions for how to behave; messages are the dialogue.

Piece	Where it goes	Job
user	<code>messages[]</code>	Current question or app input
assistant	<code>messages[]</code>	Prior model replies you choose to keep
system	<code>system</code> on <code>create</code>	Standing instructions, role, anti-hallucination rules
tools / RAG (later)	<code>create params</code> / retrieved text	Also consume the input context window

System is the closest API analogue of an “instruction file” in Claude Code / VS Code, but you must pass it **on every request** because the server is stateless. Repeating it costs input tokens; prompt cache is how you stop paying full price for a stable prefix (module 09).

Why “what is my name?” fails without history

Demo 1: user says “Hello, my name is Vijay.” Demo 2 (new `create`, only “what is 1+1?”): you get arithmetic, no name. Demo 3 (new `create`, only “what is my name?”): the model has no prior user line unless you included it. Demo 4: `messages = [user name intro, assistant greeting, user 1+1, assistant “2”, user “what is my name?”]` — now the name is in context and the model can recover it.

ChatGPT, Claude Chat, and Claude Code feel stateful because **those products** store session history and resubmit it. Your notebook is an AI application: you decide store-in-RAM, store-in-DB, or discard. Hard-coded lists are for teaching; production uses a loop that appends.

Helper methods Vijay reused

- `add_user_message(messages, text)` — append role: user.
- `add_assistant_message(messages, text)` — append the previous reply so the next call sees it.

3. `chat(messages, ...)` — call `create` with `model`, `max_tokens`, and the list; return `message.content[0].text` (or iterate blocks).

A later notebook wrapped that in a `while` loop (type until escape) as a console chatbot. Same helpers can sit behind FastAPI, Flask, Django, or Streamlit. The model does not know about your UI; it only sees the array you send.

Context versus “memory” versus RAG

In this session, **context** / **context window** means the tokens in this request: system + messages (+ tools later). Conversation “memory” is whatever *your app* persisted and put back into that window. Caching PDFs inside Claude is still not a vector database: Vijay told a questioner that querying a DB is **RAG** and is input you retrieve, not model-side long-term storage. Do not mix the three on the exam.

Prefill: assistant messages as a lead-in

You can place an assistant message *before* the model has answered, so generation continues from a seed (for example a markdown fence). That is **prefill**, taught in depth on 19 August. For this chapter: assistant role is both “history you replay” and “partial answer you want completed.”

System prompt and hallucination

Vijay’s system-prompt notebook merged optional `system` into the `params dict` passed to `create`. The purpose he emphasised: set the role and processing rules so the model is less likely to wander. It does not replace grounding with documents, tools, or RAG; it is one slice of input context.

CLASSROOM UI

How VS Code or Jupyter shows a chat widget is not in the API. Prefer **Messages API** field names if the notebook helper signatures differ.

Exam drills

- If the application does not resend prior turns, the model cannot use them.
- System prompt is a `create` parameter, not stored server-side between calls.
- User $\approx$ human message; assistant $\approx$ model (or prefilled) message in the array.
- Managing the list is application work; Claude Chat already does that for you in the product.

Official documentation

• Messages API

<https://docs.claude.com/en/api/messages>

Roles, system, and multi-turn examples.

• Prompt engineering overview

<https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>

How system versus user content should be written.

- **API overview**

<https://docs.claude.com/en/api/overview>

Client construction and request shape.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.